

1.3

Le langage SQL

NSI TERMINALE - JB DUTHOIT

1.3.1 Introduction

Nous avons étudié la structure d'une base de données relationnelle, et nous allons maintenant apprendre à :

- Créer une base de données
- créer des attributs
- Ajouter des données
- Modifier des données
- Interroger une base de données afin d'obtenir les informations souhaitées

Pour cela, il nous faut apprendre un langage de requêtes : SQL : Structured Query Language.

Nous allons utiliser le logiciel "DB Browser for SQLite" : <https://sqlitebrowser.org/>.

Définition

SQLite est une bibliothèque écrite en langage C qui propose un moteur de base de données relationnelle accessible par le langage SQL

Remarque

Noter qu'il existe d'autres systèmes de gestion de base de données relationnelle comme MySQL ou PostgreSQL. Dans tous les cas, le langage de requête utilisé est le SQL.

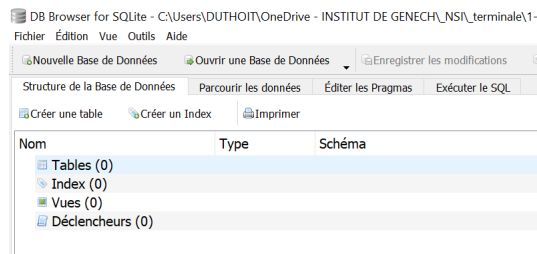
⚠ Attention, on peut parfois noter quelques petites différences de syntaxe entre les différentes SGBD).

1.3.2 Création d'une base de données

1. lancez le logiciel "DB Browser for SQLite"
2. Cliquez sur Nouvelle base de données.
3. Choisissez un nom pour cette base de données
4. Cliquez sur Annuler

☞ Un message confirme la création de la table.

La base de données est créée !



1.3.3 Création, suppression et modification d'une relation

Savoir-Faire 1.5 : Savoir créer une relation en SQL

Pour créer la relation LIVRE(id INT, titre TEXT, ann_publi INT), nous allons utiliser ici les mots clefs **CREATE TABLE** et **PRIMARY KEY** :

1. Cliquez sur l'onglet "Exécuter le SQL"
2. Recopiez le texte suivant dans la fenêtre "SQL 1"

```
CREATE TABLE LIVRE
(id INT, titre TEXT, ann_publi TEXT, PRIMARY KEY (id));
```

ou bien

```
CREATE TABLE LIVRE
(id INT PRIMARY KEY, titre TEXT, annee_publi TEXT);
```

3. Cliquez ensuite sur "Lecture" (petit triangle ou F5)

☞ Nous avons pour chaque attribut précisé son domaine : id : entier (**INT**), titre : chaîne de caractères (**TEXT**), ann__ publi : INT

☞ L'attribut "id" va jouer ici le rôle de clé primaire.

Remarque

Il faut noter la possibilité d'indiquer **NOT NULL** pour un attribut (sauf la clé primaire qui est forcément NOT NULL) ;

Cela signifie que l'on ne pourra pas créer un enregistrement avec le marqueur **NULL** pour cet attribut.

⚠ **NOT NULL** ne signifie pas différent de zéro. Cela ne signifie pas non plus un champ vide.

☛ **NULL** est un marqueur utilisé pour signifier une absence de valeur.

Il existe de nombreux type de domaine, en voici quelques-uns :

type de données	syntaxe
Entier	INT, INTEGER ...
Chaîne de caractères	TEXT
	VARCHAR(20) (au maximum 20 caractères)
	CHAR(20) (chaîne de 20 caractères, rempli de blancs si besoin)
Réel	REAL, FLOAT...

Remarque

- Type de données booléen :

SQLite ne dispose pas d'une classe de stockage booléenne distincte. Au lieu de cela, les valeurs booléennes sont stockées comme des entiers 0 (faux) et 1 (vrai).

- Date :

SQLite ne dispose pas d'une classe de stockage séparée pour stocker les dates et/ou les heures, mais SQLite est capable de stocker les dates et les heures sous forme de valeurs **TEXT**, **REAL** ou **INTEGER**.

Classe de stockage et format de date

- **TEXT** - Une date dans un format comme "AAAA-MM-JJ HH :MM :SS.SSS".
- **REAL** - Le nombre de jours depuis midi à Greenwich le 24 novembre 4714 avant J.-C.
- **INTEGER** - Le nombre de secondes depuis 1970-01-01 00 :00 :00 UTC

Vous pouvez choisir d'enregistrer les dates et les heures dans l'un de ces formats et convertir librement les formats à l'aide des fonctions intégrées de date et d'heure.

Savoir-Faire 1.6 : Modification et suppression d'une relation en SQL

Remarque

⚠ **ALTER TABLE ... MODIFY** et **ALTER TABLE ... CHANGE** n'existent pas en standard dans SQLite!! Il convient donc de l'apprendre par cœur sans pouvoir tester! C'est le seul petit inconvénient de SQLite ;-)

- Pour modifier une relation : ☛ Nous allons utiliser ici le mot clef **ALTER TABLE** :

- Pour ajouter un attribut :

```
ALTER TABLE LIVRE ADD nb_chap INT
```

- Pour modifier un attribut :

```
ALTER TABLE LIVRE MODIFY nb_chap REAL
```

- Pour changer le nom d'un attribut :

```
ALTER TABLE LIVRE CHANGE nb_chap nb_chapitres
```

- Pour supprimer un attribut :

```
ALTER TABLE AUTEUR DROP nb_chapitres
```

2. Pour supprimer une relation : 🖱 Nous allons utiliser ici le mot clef **DROP TABLE**

```
DROP TABLE LIVRE
```

⚠ A utiliser avec beaucoup d'attention, car après les données sont perdues définitivement.

1.3.4 Ajout de données

🔗 Savoir-Faire 1.7 : Savoir ajouter des données

- 🖱 Nous allons utiliser ici le mot clef **INSERT INTO** :

1. Cliquez sur "Exécuter le SQL"
2. Entrez le texte :

```
INSERT INTO LIVRE
(id, titre, ann_publi)
VALUES
(1,"Les fleurs du mal", 1857),
(2,"L'étranger", 1942),
(3,"Les misérables", 1862),
(4,"Les liaisons dangeureuses", 1782),
(5,"Le petit prince", 1943),
(6,"Cyrano de Bergerac", 1897),
(7,"Les trois mousquetaires",1844),
(8,"Le comte de Monte-Cristo",1844),
(9,"La peste", 1947),
(10,"Notre-Dame de Paris", 1831),
(11,"Orgueil et préjugés",1813),
(12,"Oliver Twist", 1839),
(13,"Le Portrait de Dorian Gray", 1890),
(14,"L'Ami retrouvé", 1971),
(15,"Gatsby le Magnifique", 1925);
```

Remarque

- | Un message nous indique une nouvelle fois que la requête a été réalisée avec succès.

Savoir-Faire 1.8 : Savoir supprimer des données en SQL

☞ Nous allons utiliser ici le mot clef **DELETE FROM** :

DELETE permet de supprimer des lignes dans une relation. On peut utiliser la clause WHERE pour sélectionner les ligne à supprimer :

```
DELETE FROM LIVRE
WHERE ann_publi = 1971 ;
```

⚠ Il est préférable de sauvegarder la base de données avant de supprimer des lignes. De cette façon, un retour en arrière sera toujours possible!

⚠ Si la clause WHERE n'est pas présente, toutes les lignes seront effacées!!

Savoir-Faire 1.9 : Savoir modifier des données en SQL

☞ Nous allons utiliser ici le mot clef **UPDATE** :

La commande UPDATE permet d'effectuer des modifications sur des lignes existantes. Exemple :

```
UPDATE LIVRE SET ann_publi = 1000
WHERE titre = "L'étranger" ;
```

Il est possible également de réaliser ceci :

```
UPDATE LIVRE SET titre = UPPER(titre)
```

1.3.5 Les requêtes d'interrogation

Savoir-Faire 1.10 : Savoir effectuer une requête d'interrogation

☞ Nous allons utiliser ici le mot clef **SELECT** :

```
SELECT id, titre, ann_publi
FROM LIVRE
```

liste des enregistrements

	id	titre	annee_publi
1	1	Les fleurs du mal	1857
2	2	L'étranger	1942
3	3	Les misérables	1862
4	4	Les liaisons dangereuses	1782
5	5	Le petit prince	1943
6	6	Cyrano de Bergerac	1897
7	7	Les trois mousquetaires	1844
8	8	Le comte de Monte-Cristo	1844
9	9	La peste	1947
10	10	Notre-Dame de Paris	1831
11	11	Orgueil et préjugés	1813
12	12	Oliver Twist	1839
13	13	Le Portrait de Dorian Gray	1890
14	14	L'Ami retrouvé	1971
15	15	Gatsby le Magnifique	1925

Il est possible de saisir :

```
SELECT *
FROM LIVRE
```

pour obtenir tous les attributs !

☞ * permet d'obtenir tous les attributs.

L'utilisation de la commande **SELECT** en SQL peut potentiellement afficher des lignes en double.

☞ Pour éviter des redondances dans les résultats il faut simplement ajouter **DISTINCT** après le mot **SELECT**.

☞ Nous allons utiliser ici le mot clef **WHERE** pour filtrer les résultats :

⚠ Il est possible de combiner les conditions à l'aide d'un **OR** ou d'un **AND**

Il est aussi possible de trier les résultats :

☞ Nous allons utiliser ici le mot clef **ORDER BY**

rajouter la clause **ORDER BY** afin d'obtenir les résultats classés dans un ordre précis.

```
SELECT titre
FROM LIVRE
WHERE ann_publi >= 1900 ORDER BY titre
```

	titre
1	Gatsby le Magnifique
2	L'étranger
3	La peste
4	Le petit prince
5	L'Ami retrouvé

Remarque

☞ Il est possible d'obtenir un classement en sens inverse à l'aide de la clause **DESC**

```
SELECT titre, ann_publi
FROM LIVRE
WHERE ann_publi >= 1900 ORDER BY ann_publi DESC
```

Exercice 1.10

Écrire une requête qui permet d'obtenir la liste des livres qui ont été écrits après 1950.

	id	titre	annee_publi
1	14	L'Ami retrouvé	1971

Exercice 1.11

Écrire une requête qui permet d'obtenir la liste "titre, annee__ publication", liste rangée par ordre chronologique de publication, pour les livres écrits après 1900.

	titre	annee_publi
1	L'Ami retrouvé	1971
2	La peste	1947
3	Le petit prince	1943
4	L'étranger	1942
5	Gatsby le Magnifique	1925

Eviter les doublons

☛ Nous allons utiliser ici le mot clef **DISTINCT**

Si on effectue :

```
SELECT ann_publi  
FROM LIVRE
```

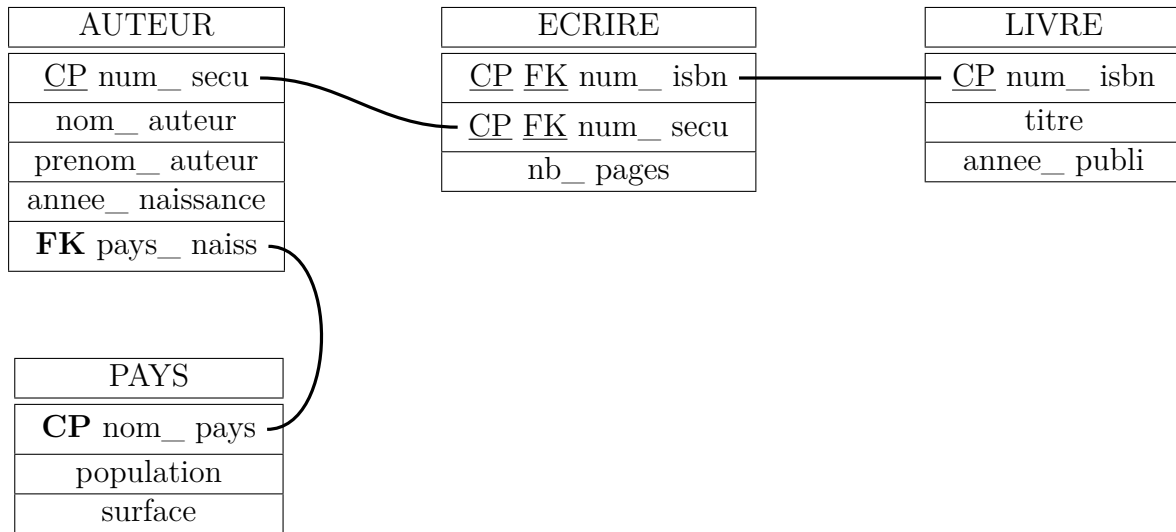
On se retrouve avec des doublons (pour 1844 par exemple).

Il suffit alors d'utiliser **DISTINCT** :

```
SELECT DISTINCT ann_publi  
FROM LIVRE
```

1.3.6 Lier plusieurs tables d'une même base de données

L'objectif ici est de créer le schéma relationnel suivant, d'y ajouter des enregistrements pour ensuite effectuer des requêtes !!!



Savoir-Faire 1.11 : Savoir définir une clé étrangère

- Nous allons utiliser ici le mot clef **FOREIGN KEY** notamment.
- Pour créer la clef étrangère sur l'attribut `pays_naiss` de la relation `AUTEURS`, on écrira :

```

CREATE TABLE AUTEUR
(num_secu INT,
nom_auteur TEXT,
prenom_auteur TEXT,
annee_naissance INT,
pays_naissance TEXT,
PRIMARY KEY (num_secu),
FOREIGN KEY (pays_naissance) REFERENCES PAYS(nom_pays));
  
```

Exercice 1.12

Faire de même avec la table `PAYS` et la table `LIVRE`.

Exercice 1.13

1. Compléter les instruction SQL quivantes afin d'ajouter le livre "Le Cercle littéraire des amateurs d'épluchures de patates" :

```

INSERT INTO LIVRE
VALUES
(...);
  
```

2. Ajouter les auteurs concernés :

```

INSERT INTO AUTEUR
(num_secu, nom_auteur, prenom_auteur, annee_naiss, pays_naiss)
VALUES
(1, 'Baudelaire', 'Charles', 1821, 'France'),
(2, 'Camus', 'Albert', 1913, 'Algérie'),
...
  
```

3. Entrer aussi les informations concernant la relation ECRIRE :

```
INSERT INTO ECRIRE
(id_livre, id_auteur, nb_page)
VALUES
(1, 1, 100),
(2, 2, 100),
...
```

(on indiquera de façon arbitraire un nb non nul de pages)

4. Et enfin, on entrera les informations pour la relation PAYS :

```
INSERT INTO PAYS
(nom_pays, population, superficie)
VALUES
('France', 67, 643),
('Irlande', 5, 70),
...
```

On exprimera la population en million d'habitants et la superficie en milliers de km^2



Savoir-Faire 1.12 : Savoir effectuer des jointures

☞ Nous allons utiliser ici le mot clef **JOIN** :

Observez bien cet exemple qui permet de joindre les différentes tables et d'afficher le nom, prénom et année de naissance des auteurs nés en France

```
SELECT nom_auteur, prenom_auteur, annee_naiss
FROM LIVRE
JOIN ECRIRE ON ECRIRE.id_livre = LIVRE.id_livre
JOIN AUTEUR ON ECRIRE.id_auteur = AUTEUR.id_auteur
JOIN PAYS ON PAYS.nom_pays = AUTEUR.pays_naiss
WHERE pays_naiss = "France"
```

	nom_auteur	prenom_auteur	annee_naiss
1	Baudelaire	Charles	1821
2	Hugo	Victor	1802
3	Laclos	Choderlos de	1741
4	Saint-Exupéry	Antoine de	1900
5	Rostand	Edmond	1868
6	Dumas	Alexandre	1802
7	Dumas	Alexandre	1802
8	Hugo	Victor	1802



Exercice 1.14

Faire les requêtes qui permettent de répondre aux requêtes suivantes, sur l'ensemble des 4 tables précédentes : (Vous trouverez la réponse attendue issue de la requête)

1. Quel est le pays de naissance d'un auteur qui a publié un livre en 1925

	nom_pays
1	USA

2. Quel auteur à écrit "Notre-Dame de Paris" ?

	nom_auteur
1	Hugo

3. Quels auteurs ont écrit "Le cercle littéraire des amateurs d'épluchures de patates" ?

	nom_auteur
1	Barrows
2	Shaffer

4. Quels sont les livres écrits par "Camus" ?

	titre
1	L'étranger
2	La peste

1.3.7 Les fonctions d'agrégation

Les fonctions d'agrégation dans le langage SQL permettent d'effectuer des opérations statistiques sur une colonne. Les principales fonctions sont :

sum : pour calculer la somme d'un attribut

min : pour récupérer la valeur minimale

max : pour récupérer la valeur maximale

avg : pour calculer la moyenne

count : pour compter le nombre d'enregistrements

Savoir-Faire 1.13 : Savoir utiliser les fonctions d'agrégation

| 📌 Les fonctions d'agrégation seront utilisées dans l'exemple suivant !